

Homework 3

Name: YOUR NAME; Collaborator: YOUR COLLABORATOR

Important: For this homework and all subsequent homeworks, whenever you are asked to analyze a particular algorithm or to design an algorithm, your solution must include a **clear English description** of your thought process. Remember, your audience is a human, not a computer, and we want to understand the process you used to arrive at the solution. This way, we can provide appropriate feedback.

1. This question is a follow-up on problem 2 from Homework 2. You are a data engineer working at the digital advertisement firm which runs massive data campaigns. After problem 2, you reported your Bloom Filter findings to the client. They were impressed, but the engineering team has a new constraint. For the next campaign, you still need to estimate $n_0 \in \mathbb{N}$, the distinct number of people that saw the ad. The ad campaign can only devote a tiny amount of memory for your algorithm—still, they need an estimate the number of distinct viewers n_0 . You will suggest a new hashing-based approach:
 - You will consider a hash function $\mathbf{h}: \mathcal{U} \rightarrow [0, 1]$ which is uniformly random. $\mathcal{U}$ is the finite universe (i.e., the set of all possible IP addresses), and for every $x \in \mathcal{U}$, $\mathbf{h}(x)$ is a uniformly random number in $[0, 1]$.
 - Your algorithm will maintain a single number $\mathbf{z}$, originally set to 1. When a new IP address, corresponding to $x \in \mathcal{U}$ sees the ad, you update the number $\mathbf{z} \leftarrow \min\{\mathbf{z}, \mathbf{h}(x)\}$. Importantly, if $x \in \mathcal{U}$ sees the ad once, or *twice*, the algorithm behaves the same way!

Suppose the true (unknown) set of distinct IP addresses who saw the add is $X \subset \mathcal{U}$, and let $L = |X|$ be the true number which the algorithm wishes to learn. The goal of this problem is to understand how one may use $\mathbf{z}$ to estimate L , if we run this algorithm throughout the entire campaign.

- (a) Let $t \in [0, 1]$. Compute $\Pr[\mathbf{h}(\mathbf{x}) \geq t]$. Use this to compute the $\Pr[\mathbf{z} \geq t]$. The expression should be in terms of t and L .
- (b) Note that $\mathbf{z}$ is a continuous random variable. Let $\Psi(\cdot)$ be its p.d.f. What is $\Psi(t)$? (Hint: recall the relation between cdf and pdf and use (i)).
- (c) Now, compute $\mathbf{E}[\mathbf{z}]$ using $\Psi(t)$. (Hint: You may use the fact $\int_0^1 x^j = 1/(j+1)$, and we encourage you to proceed by substitution to simplify the expression.)
- (d) Prove that $\text{Var}(\mathbf{z}) \leq \frac{1}{(L+1)^2}$.

With a thorough understanding of the random variable $\mathbf{z}$, you are now ready to design and analyze the estimator. Suppose you instantiated $k = 10/\epsilon^2$ independent copies of the above algorithm, and after obtaining $\mathbf{z}_1, \dots, \mathbf{z}_k$, define $\mathbf{Z} = \frac{1}{k} \sum_{i=1}^k \mathbf{z}_i$.

- (e) Use Chebyshev's inequality to argue that with probability $9/10$,

$$\frac{1-\epsilon}{L+1} \leq \mathbf{Z} \leq \frac{1+\epsilon}{L+1}.$$

- (f) Conclude that $1/\mathbf{Z}$ is a good estimate for L with probability at least $9/10$. For $\epsilon = 0.05$ (i.e., wanting to determine the number of distinct IP addresses up to %5 error), assume each $\mathbf{z}_k$ uses 8 bytes of storage, how much memory does your algorithm use (in kilobytes).
2. You are working on the backend of a large-scale web crawler. Your system processes billions of web pages and needs to identify “near-duplicates” to prevent storing redundant copies of the same content. Your system represents web pages using d -dimensional vectors $x \in \{-1, 1\}^d$ and uses the cosine similarity, so you have decided to use SimHash as a Locality Sensitive Hashing (LSH) technique. You recall from lecture that the theoretical derivation of SimHash takes $\mathbf{h}(x) = \text{sign}(\langle x, \mathbf{z} \rangle)$ where $\mathbf{z}$ is a random vector drawn along a *uniformly random direction*—but how does one generate such a vector $\mathbf{z}$? The standard normal random variable $\mathcal{N}(0, 1)$ has density $p(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$, and the `numpy` library in python provides functionality to generate normal random variables (via `numpy.random.randn` function).

- (a) Let $\mathbf{g}_1, \dots, \mathbf{g}_d$ be d independent draws of $\mathcal{N}(0, 1)$. Prove that the vector

$$\mathbf{z} = \frac{(\mathbf{g}_1, \dots, \mathbf{g}_d)}{\sqrt{\mathbf{g}_1^2 + \dots + \mathbf{g}_d^2}} \in \mathbb{R}^d$$

is a unit vector distributed uniformly in the d -dimensional sphere. (Hint: Prove that the pdf of the random *vector* $(\mathbf{g}_1, \dots, \mathbf{g}_d)$ —recall, independence implies this is the product of individual p.d.fs—only depends on the Euclidean norm of the point (i.e., it’s distance away from 0). Now, what can you say about the length of any draw from $\mathbf{z}$).

As a result of part (a), you decide to use Gaussian random vectors in order to generate $\mathbf{h}$, but you quickly realize that, Gaussian random numbers use a lot of numerical precision, and performing precise floating-point multiplications in order to compute $\mathbf{h}(x)$ is too slow for your latency requirements. You investigate whether you could simplify SimHash to be $\mathbf{h}(x) = \text{sign}(\langle x, \mathbf{z} \rangle)$ where $\mathbf{z} \in \{-1, 1\}^d$ is drawn uniformly at random (needing only d bits!).

- (b) Suppose $x, y \in \{-1, 1\}^d$ are any two web pages in your system, and we will sample $\mathbf{z} \sim \{-1, 1\}^d$ to define the SimHash. For $k \geq 0$, let $\mathbf{Z}_k$ be the sum of k independent and unbiased ± 1 random variables. Prove

$$\Pr[\mathbf{h}(x) = \mathbf{h}(y)] = \Pr[|\mathbf{Z}_a| \geq |\mathbf{Z}_\ell|],$$

where $a \in \mathbb{N}$ is the number of coordinates where x and y agree, and $\ell \in \mathbb{N}$ is the number of coordinates where x and y disagree.

- (c) Prove that, if $\ell = 1$, then $\Pr[\mathbf{h}(x) \neq \mathbf{h}(y)] = O(1/\sqrt{d})$, where you may use the fact that $\binom{r}{(r-j)/2} = 2^r \cdot O(j/\sqrt{r})$.